

Tập 24 - BPF Maps

BPF Maps: Hash, LRU, Array, Per-CPU — Vũ khí hiệu năng của Cilium

Phần 3 — Cilium · `#ebpf` `#bpfmaps` `#kernel` `#hashmap` `#performance`



Mục tiêu tập này

- BPF Maps là gì — cầu nối giữa kernel space và user space
- 4 loại Map quan trọng nhất trong Cilium
- Tại sao BPF Maps thay thế được iptables chains
- Inspect BPF Maps thực tế để hiểu Cilium đang "nghĩ" gì

Prerequisites: Cilium đang chạy trên cluster (từ Tập 23)

BPF Maps là gì?

BPF Maps = Shared memory giữa:



Ví dụ thực tế:

cilium-agent ghi policy vào BPF Map

→ BPF program trong kernel đọc Map per-packet

→ Quyết định ALLOW/DROP trong nanoseconds

Không cần chuyển đổi ngữ cảnh (Context Switch):

– Thông thường: Kernel phải chuyển tiếp về User Space để hỏi Agent quyết định.

– Với BPF Maps: eBPF program tự đọc Map trực tiếp ngay trong Kernel.

→ Tiết kiệm CPU tối đa, xử lý gói tin trong vài nanoseconds!

Tại sao eBPF cần BPF Maps?

Do cơ chế an toàn của Linux Kernel, eBPF program bị giới hạn rất ngặt nghèo:

- **Stateless (Không lưu trạng thái):** Mỗi khi một packet đi qua, eBPF program được gọi, chạy xong là kết thúc. Nó không thể tự lưu biến toàn cục để nhớ packet trước đó.
- **Không truy cập trực tiếp bộ nhớ User Space:** Kernel space và User space tách biệt. Không thể dùng con trỏ (pointer) để đọc/ghi chép chung.

☞ **BPF Maps giải quyết cả hai vấn đề:**

1. Là **Stateful Storage** giúp eBPF program lưu trữ trạng thái (ví dụ: conntrack, metrics).
2. Là kênh **IPC (Inter-Process Communication)** để Kernel và User space trao đổi thông tin cấu hình và dữ liệu cực nhanh.

Cơ chế hoạt động & Vòng đời của BPF Maps

1. Khởi tạo (Cilium-Agent hoặc Loader tạo Map trong Kernel qua syscall bpf())



2. Sử dụng (Helper functions trong Kernel Program / CLI ở User Space)

- bpf_map_lookup_elem(&map, &key)
- bpf_map_update_elem(&map, &key, &value, flags)
- bpf_map_delete_elem(&map, &key)



3. Lưu giữ (Pinned vào Virtual Filesystem: /sys/fs/bpf/)

- Map KHÔNG mất đi khi eBPF program dừng hoặc Cilium Agent restart!
- Đảm bảo data / policy không bị gián đoạn (zero-downtime).

4 loại Map quan trọng

Type	Use case	Đặc điểm
BPF_MAP_TYPE_HASH	Policy lookup: IP → rule	Tìm kiếm O(1) siêu tốc, chống va chạm băm (collision), kích thước động
BPF_MAP_TYPE_LRU_HASH	Conntrack: flow state	Kích thước cố định, tự động giải phóng kết nối cũ nhất khi đầy (tránh tràn/treo)
BPF_MAP_TYPE_ARRAY	Config, metrics counters	Truy cập qua chỉ số (Index), bộ nhớ cấp phát cứng từ đầu, tốc độ cao nhất
BPF_MAP_TYPE_PERCPU_HASH	Per-CPU packet counters	Mỗi CPU core một vùng nhớ riêng, không tranh chấp khóa (lock-free)

Hash Map: Policy Lookup O(1)

`cilium_policy_<endpoint_id> (BPF_MAP_TYPE_HASH)`

Key: {src_ip, dst_ip, dst_port, protocol}

Value: {verdict: ALLOW/DROP, action_flags}

Lookup O(1) nghĩa là gì?

- Dù có 10 hay 100.000 luật policy, thời gian tìm kiếm vẫn không đổi (chỉ mất 1 lần tính toán hash và 1 lần đọc bộ nhớ).
- Khác hoàn toàn với iptables phải duyệt tuyến tính O(N) từ trên xuống dưới.

Khi packet đến:

1. BPF program trích xuất thông tin (IP, Port, Protocol) từ packet header
2. `bpf_map_lookup_elem(&cilium_policy, &key)` (Tìm kiếm cực nhanh)
3. Trả về ALLOW → Chuyển tiếp gói tin
Trả về NULL → DROP (Từ chối theo cơ chế default deny)

LRU Hash Map: Conntrack không cần lock

`cilium_ct4_glob / cilium_ct_tcp4 (BPF_MAP_TYPE_LRU_HASH)`

Key: {src_ip, src_port, dst_ip, dst_port, proto}

Value: {state, last_seen, flags, rev_nat_index}

(Tên thực tế bị rút ngắn thành 15 ký tự do giới hạn của Kernel)

Giải thích thuật ngữ:

- Conntrack (Connection Tracking): Theo dõi trạng thái kết nối TCP/UDP.
- LRU (Least Recently Used): Tự động xóa kết nối cũ nhất khi bảng đầy (512K kết nối).
→ Không gây nghẽn mạng hay crash hệ thống.

So sánh với cơ chế mặc định của Linux (nf_conntrack):

- nf_conntrack: Dùng khóa (spinlock) khi cập nhật → Gây tranh chấp (contention) giữa các CPU core, làm nghẽn hệ thống khi lưu lượng traffic cực lớn.
- BPF LRU: Thiết kế lockless (per-CPU) giúp hệ thống scale tuyến tính với số CPU.

Array Map: Config & Tail Calls siêu tốc

`cilium_runtime_config (BPF_MAP_TYPE_ARRAY)`

Key: Index (chỉ số mảng: 0, 1, 2, ...)

Value: Config value / state / program fd

Đặc điểm:

- Cố định số phần tử (Fixed-size, pre-allocated)
- Không thể xóa phần tử (chỉ có thể ghi đè)
- Tốc độ lookup cực đại (chỉ là offset pointer arithmetic)

Ứng dụng trong Cilium:

1. Lưu Config: eBPF program đọc nhanh trạng thái bật/tắt của tính năng (ví dụ: Enable IPsec?, NodePort?, MTU size).
2. BPF_MAP_TYPE_PROG_ARRAY (Tail Calls): Chứa danh sách eBPF programs.
→ BPF program này nhảy sang BPF program khác không tốn overhead!

Per-CPU Hash Map: Counters lock-free

`cilium_metrics (BPF_MAP_TYPE_PERCPU_HASH)`

Tại sao cần Per-CPU Map cho Metrics?

- Nếu tất cả CPU core cùng cộng dồn vào một biến đếm chung (global counter), chúng sẽ phải tranh chấp khóa (lock) để ghi dữ liệu, làm chậm luồng mạng.
- Giải pháp Per-CPU: Mỗi CPU core sở hữu 1 bộ đếm riêng biệt trong bộ nhớ.
 - Không cần lock, không cần atomic operation, tốc độ xử lý tối đa!

Cách hoạt động:

1. eBPF program chạy trên CPU 0 chỉ ghi vào bộ đếm của CPU 0.
2. Khi Cilium Agent (User Space) cần hiển thị tổng số gói tin:
 - Đọc bộ đếm từ tất cả các CPU core [`val_cpu0`, `val_cpu1`, ...] và cộng tổng lại.

Lab Time: Inspect BPF Maps trực tiếp

Chúng ta sẽ thực hành:

1. **List BPF maps:** `bpftool map list` để thấy tất cả maps Cilium tạo.
2. **Xem conntrack table:** `cilium bpf ct list global` — active connections.
3. **Xem policy map:** `cilium bpf policy list` — rules đang enforce.
4. **Demo O(1) vs O(n):** So sánh lookup time iptables vs BPF hash.

👉 Hãy làm theo các bước chi tiết trong file `lab-guide.md`

Tập tiếp theo (Tập 25): Kiến trúc Cilium — Operator, Agent, GoBGP, Hubble so sánh với Calico.